MAXAM | FANEXA

MANUAL TÉCNICO

Backend

Sistema de Control de Viáticos (SCV)

Gestión de viajes corporativos y rendición de gastos

Autor: Mateo Alejandro Merino Vidal

Empresa: MAXAM FANEXA

Componente: API REST (Node.js, Express y Supabase)

Versión: 2.1

Fecha: 24 de octubre de 2026

Índice

1. Introducción **3**
1.1. Propósito del documento . . . 3
1.2. Descripción general del sistema . . . 3
1.3. Alcance . . . 3

2. Stack tecnológico **3**

3. Arquitectura **4**
3.1. Capas . . . 4
3.2. Estructura del proyecto . . . 5

4. Instalación y configuración **5**
4.1. Requisitos . . . 6
4.2. Instalación . . . 6
4.3. Variables de entorno . . . 6

5. Base de datos **7**
5.1. Scripts . . . 7
5.2. Tablas . . . 7
5.3. Roles . . . 9
5.4. Decisiones de diseño . . . 9

6. Autenticación y sesión **9**
6.1. Tokens . . . 10
6.2. Validación en cada petición . . . 10
6.3. Cambio de contraseña obligatorio . . . 10
6.4. Límites de intentos . . . 11

7. Autorización **11**
7.1. Por rol y por cargo . . . 11
7.2. Asignación por jerarquía . . . 11
7.3. Acceso a un viaje . . . 11
7.4. Observaciones . . . 11

8. Flujos de aprobación **12**
8.1. Reglas de transición . . . 12

9. Reglas de negocio **12**
9.1. Monto del viaje . . . 12
9.2. Control de gasto diario . . . 13
9.3. Moneda en viajes internacionales . . . 13
9.4. Tipo de cambio . . . 13
9.5. Retenciones impositivas . . . 13
9.6. Plazos de registro . . . 14
9.7. Rendición por terceros . . . 14

10. Catálogo de endpoints **14**

10.1. Autenticación y usuarios . . . 14
10.2. Viajes, gastos y facturas . . . 15
10.3. Revisión y aprobación . . . 16
10.4. Solicitudes . . . 17
10.5. Catálogos y administración . . . 17

11. Servicios transversales **17**

12. Documentos y notificaciones **18**
12.1. Documentos PDF . . . 18
12.2. Correos inmediatos . . . 19
12.3. Resumen de pendientes . . . 19

13. Extracción de datos de comprobantes **20**

14. Seguridad **20**

15. Despliegue **21**
15.1. Piloto en Render . . . 21
15.2. Servidor VPS . . . 21

16. Dependencias y seguridad **22**

17. Pruebas automáticas **22**
17.1. Integración continua . . . 22

18. Convenciones **23**

1. Introducción

1.1 Propósito del documento

El presente manual describe la construcción interna del backend del Sistema de Control de Viáticos (SCV) de MAXAM FANEXA. Está dirigido a desarrolladores y personal técnico que necesiten instalar, mantener, extender o desplegar la API, y documenta su arquitectura, modelo de datos, reglas de negocio, seguridad, catálogo de endpoints y procedimientos de despliegue.

1.2 Descripción general del sistema

El SCV gestiona el ciclo completo de un viaje corporativo: el empleado registra la solicitud de viaje, que pasa por la revisión de su supervisor, la aprobación del aprobador y la asignación de fondos por tesorería. Con el viaje en curso, el empleado registra sus gastos (facturas, recibos y gastos sin factura) y, al finalizar, los envía a una segunda cadena de revisión: supervisor, aprobador (solo si hay bebidas alcohólicas) y revisor final. El backend genera los documentos oficiales (memorándum, confirmación de fondos, recibos, rendición final y planilla en PDF), envía las notificaciones por correo y calcula las retenciones impositivas de cada gasto.

1.3 Alcance

El backend expone una API REST consumida exclusivamente por el cliente web del sistema. No ofrece servicios a terceros. Toda la persistencia se realiza en una base de datos PostgreSQL alojada en Supabase, a la que el backend accede con la clave de servicio.

2. Stack tecnológico

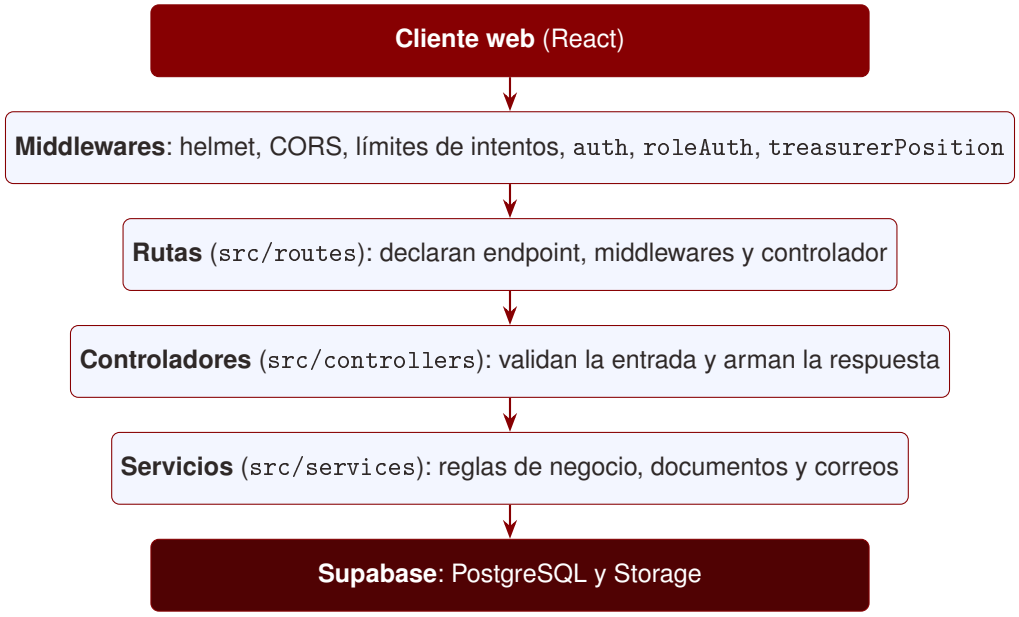
Tecnología	Propósito
Node.js 22.12+ y Express 5	Servidor HTTP y enrutamiento de la API
Supabase (PostgreSQL)	Base de datos relacional, almacenamiento de imágenes y cliente JavaScript
jsonwebtoken	Emisión y verificación del token de acceso y del token de renovación
bcrypt	Cifrado de contraseñas
cookie-parser	Lectura de la cookie `httpOnly` del token de renovación
helmet y cors	Cabeceras de seguridad y control de orígenes permitidos
express-rate-limit	Límite de intentos de ingreso, recuperación de contraseña y extracción de facturas
multer	Recepción de imágenes de comprobantes y fotos de perfil
Puppeteer 25	Conversión de HTML a PDF (memorándum, recibos, rendiciones); se carga con `import()` al generar el primer PDF

Tecnología	Propósito
@google/generative-ai	Modelo Gemini para extracción de comprobantes y detección de alcohol
sib-api-v3-sdk (Brevo)	Envío de correo transaccional
node-cron	Resumen de pendientes por correo en los días y horas que configura el administrador
Jest y Supertest	Pruebas unitarias y de la API
GitHub Actions	Ejecución automática de las pruebas en cada envío al repositorio

3. Arquitectura

3.1 Capas

El backend sigue una arquitectura en capas. Cada petición atraviesa los middlewares de seguridad, llega a una ruta, que delega en un controlador; el controlador valida y traduce la petición HTTP y llama a un servicio, que concentra la lógica de negocio y el acceso a datos.



Capa	Responsabilidad	Restricción
Ruta	Asocia método, URL, middlewares y controlador	No contiene lógica
Controlador	Lee parámetros, valida formato, responde con el código HTTP	No accede a la base de datos directamente

Capa	Responsabilidad	Restricción
Servicio	Reglas de negocio, consultas, documentos, correos	Devuelve datos o `{error, status}`, nunca responde HTTP

Los servicios devuelven siempre un objeto con el resultado o con `error` y `status`; el controlador decide la respuesta:

```
const approveTrip = async (req, res) => {
  const result = await treasurerService.approveTrip(req.params.tripId, req.user.id_usuario)
  if (result.error) {
    return res.status(result.status || 500).json({error: result.error})
  }
  else {
    return res.json({message: result.message})
  }
}
```

3.2 Estructura del proyecto

```
src/
|-- config/supabase.js       Cliente de conexión
|-- routes/                  Declaración de rutas
|   |-- admin/                 admin
|   |-- approval/              review, reviewer, approver, treasurer,
|   |                          deadlineAuthorization, substitution
|   |-- catalog/               role, position, section, expenseCategory, tax, audit
|   |-- expense/               expense, invoice, supplier
|   |-- trip/                  trip
|   `-- user/                  user, auth
|-- controllers/             Misma división que routes
|-- services/
|   |-- approval/              revisión, aprobación, tesorería, documentos
|   |-- catalog/               cargos y secciones
|   |-- expense/               gastos, facturas, recibos, extracción
|   |-- trip/                  viajes, acceso, observaciones, planilla, exportación
|   |-- user/                  tokens, usuarios, importación, directorio
|   `-- shared/                correo, PDF, plazos, jerarquía, IA, resumen diario
|-- middlewares/             auth, roleAuth, treasurerPosition, límites de intentos
|-- utils/                   escape HTML, importes en letras, código de viaje, retenciones
|-- app.js                   Configuración de Express (seguridad, CORS, rutas)
`-- index.js                 Arranque del servidor y del cron de resúmenes
scripts/sendDigest.js        Envío manual del resumen de pendientes
scripts/cleanStorage.js      Borra los archivos que ya no usa nadie
database/                    Scripts SQL del esquema
tests/                       Pruebas: services/ (unitarias), api/ (de la API)
.github/workflows/           Pruebas automáticas en GitHub Actions
```

4. Instalación y configuración

4.1 Requisitos

- Node.js 22.12 o superior y npm.
- Un proyecto de Supabase con el esquema aplicado (sección 5).
- Cuentas y claves de Brevo (correo), Google Gemini (lectura de comprobantes) y AbstractAPI (validación de correos).
- Las librerías del sistema que requiere el Chrome de Puppeteer. `npm ci` lo descarga en `.cache/puppeteer` dentro del proyecto (`.puppeteerrc.cjs`), para que Render lo conserve.

4.2 Instalación

```
npm install
cp .env.example .env      # completar las variables
npm run dev               # desarrollo con recarga automática
npm start                 # producción
npm test                  # pruebas unitarias
npm run test:api          # pruebas de la API (base de prueba)
npm run limpiar-archivos -- --prueba   # lista los archivos sueltos
npm run resumen -- --prueba   # simula el resumen de pendientes
```

4.3 Variables de entorno

Variable	Propósito
`PORT`	Puerto del servidor (por defecto 5000)
`NODE_ENV`	`production` marca la cookie de sesión como `secure` y `sameSite=none`
`SUPABASE_URL`	Dirección del proyecto de Supabase
`SUPABASE_KEY`	Clave `service_role` (acceso completo; nunca se publica)
`JWT_SECRET`	Firma del token de acceso
`JWT_REFRESH_SECRET`	Firma del token de renovación
`GEMINI_API_KEY`	Extracción de comprobantes y detección de alcohol
`BREVO_API_KEY`	Envío de correos
`ABSTRACT_EMAIL_API_KEY`	Verifica que un correo exista antes de enviar el código de recuperación
`FRONTEND_URL`	Enlace del botón “Ingresar al sistema” de los correos
`CORS_ORIGINS`	Opcional. Dominios del cliente permitidos, separados por coma
`TRUST_PROXY`	Opcional. `true` detrás de un proxy (Nginx); en Render se detecta solo
`PUPPETEER_EXECUTABLE_PATH`	Opcional. Ruta a un Chromium propio

Variable	Local	Piloto (Render)	VPS
NODE_ENV	development	production	production
FRONTEND_URL	http://localhost:5173	Dominio de Vercel	Dominio final
TRUST_PROXY	No se define	No se define	true

5. Base de datos

5.1 Scripts

La carpeta database/ contiene los scripts para levantar la base en Supabase, en este orden:

Archivo	Contenido
01_schema.sql	Las 24 tablas, relaciones e índices
02_functions.sql	Función incrementar_correlativo_recibo() para numerar recibos
03_seed.sql	Roles, cargos, impuesto IVA, categorías de gasto y configuración de recordatorios
04_rls_hardening.sql	Activa seguridad por fila (RLS) en todas las tablas
testing_00_borrar_todo.sql	Elimina las 24 tablas (solo pruebas)
testing_01_limpieza_completa.sql	Borra datos operativos, conserva catálogos
testing_02_organizacion_prueba.sql	Organización de prueba con jerarquía completa; la usan también las pruebas automáticas

5.2 Tablas

Tabla	Descripción
Rol	Roles del sistema (identificadores fijos)
Cargo	Cargos con su viático diario en bolivianos y dólares
Seccion	Secciones de la empresa; respaldo en la jerarquía de revisión. Guarda su código de Oracle (codigo_seccion, 3 dígitos) para la cuenta del gasto; la dependencia va en el usuario (Usuario.codigo_dependencia) y se toma del jefe directo del que viaja, o la suya si no tiene jefe; el backend los valida y solo acepta los campos editables
Usuario	Usuarios, jefe directo, estado de contraseña y sesión
Codigo_Verificacion	Códigos de recuperación de contraseña (5 minutos)

Tabla	Descripción
`Viaje`	Viajes, estado, revisores asignados, monto asignado en Bs y USD (calculado con las tarifas del cargo) y ciclo de revisión
`Solicitud_Autorizacion_Plazo`	Ampliaciones de plazo; las pide el titular o su reemplazo
`Solicitud_Reemplazo`	Solicitudes para que otra persona rinda el viaje
`Categoria_Gasto`	Plan de cuentas de la empresa (112 cuentas de la planilla de Castro más OTROS), con la cuenta contable de Oracle en el nombre y una nota de cuándo usarla (`descripcion`)
`Proveedor`	Emisores de facturas
`Gasto`	Gastos con sus retenciones impositivas y, en los sin factura, el número impreso del recibo en papel (`numero_recibo`, opcional)
`Gasto_Subitem`	Desglose de un gasto en varios conceptos
`Gasto_Tramo_Moneda`	Conversión a USD de un gasto internacional en otra moneda (monto original, tipo de cambio y equivalente)
`Imagen`	Comprobante asociado a un gasto
`Imagen_Respaldo`	Hasta 2 respaldos extra de un gasto internacional, para el comprobante del cambio de moneda; se borran junto con el gasto
`Factura`	Datos de la factura de un gasto, con el CUF (código de autorización) que pide Oracle
`Detalle_Factura`	Productos de cada factura; la cantidad admite hasta 3 decimales y el precio unitario hasta 4
`Impuesto` y `Factura_Impuestos`	Catálogo de impuestos y su relación con facturas
`Comentario`	Observaciones de revisión y justificaciones de exceso
`Auditoria`	Ingresos, salidas y cambios de contraseña
`Correlativo_Recibo` y `Recibo`	Numeración de recibos: cada gasto sin factura recibe su número al registrarlo (`receiptNumberService`, `enviado` falso hasta mandarlo) y es el N° DOCUMENTO de la planilla cuando no hay recibo original (si lo hay, va el original), marcado en lila; si el gasto tiene número de recibo en papel, el recibo del sistema muestra los dos (“Recibo Nº . . . · Recibo original Nº . . . ”) y el “Son” va en letras con el importe a cancelar
`Correlativo_Rendicion`	Último número de rendición de cada mes; el trigger `viaje_numero_rendicion` asigna el siguiente al viaje (`numero_rendicion`, `periodo_rendicion`) cuando queda aprobado del todo, según el mes de su último comprobante
`Revision_Viaje`	Historial de aprobaciones y rechazos por persona y etapa
`Configuracion_Recordatorio`	Una sola fila con los días y horas del resumen de pendientes

5.3 Roles

Id	Rol	Función
1	ADMINISTRADOR	Usuarios, cargos, secciones, historial de accesos y resumen general
2	SUPERVISOR	Revisión previa del viaje y revisión de la rendición
3	EMPLEADO	Registra viajes y gastos
4	REVISOR	Revisión final, ampliaciones de plazo y reemplazos (rol único)
5	APROBADOR	Aprobación del viaje y revisión adicional por alcohol (rol único)
—	Tesorero	No es un rol: se identifica por el cargo “asistente de caja y tesorería”

5.4 Decisiones de diseño

- **Marcas temporales** en `timestamptz`: el tipo sin zona horaria desplazaba las fechas cuatro horas respecto de Bolivia.
- **Retenciones**: `base_imponible`, `retencion_rc_iva`, `retencion_iue`, `retencion_it` e `importe_costo` se guardan al registrar como referencia, pero las pantallas, la planilla y los recibos las calculan siempre desde lo pagado con `utils/retentions.js`, para que ningún gasto muestre otro cálculo.
- **Orden de los gastos**: las consultas del detalle de un viaje, las revisiones y las planillas ordenan los gastos por fecha y, dentro del mismo día, por orden de registro.
- **Moneda**: los gastos en bolivianos se guardan con `BOB`; los internacionales con la moneda en mayúsculas.
- **Cantidades y precios**: `Detalle_Factura.cantidad` es `numeric(12, 3)` para admitir litros o kilos con hasta tres decimales, y `Detalle_Factura.precio` es `numeric(12, 4)` para guardar el precio unitario como figura en el papel (por ejemplo 3.7456 Bs por litro). Los montos de dinero siguen con dos decimales.
- **Motivo del viaje**: `Viaje.motivo` admite 500 caracteres; los asuntos de correo lo acortan con `shortenText`.
- **Observaciones**: `Comentario.id_gasto` usa `on delete set null`; al borrar un gasto la observación se conserva.
- **Solicitudes cerradas solas**: si el viaje pasa a revisión con una solicitud pendiente, queda `RECHAZADA` sin `id_revisor` y con el motivo del cierre.
- **RLS activo**: el backend usa la clave de servicio; RLS impide el acceso directo con la clave pública.

6. Autenticación y sesión

6.1 Tokens

Token	Duración	Ubicación y uso
Acceso	15 minutos	En memoria del cliente; cabecera `Authorization: Bearer`
Renovación	7 días	Cookie `httpOnly`; solo la lee `POST /auth/refresh`

El token de acceso incluye `id_usuario`, `email_corporativo`, `id_rol` y el motivo de cambio de contraseña pendiente. Al vencer, el cliente pide uno nuevo a `/auth/refresh`; si la cookie venció, fue invalidada o el usuario está suspendido, la cookie se borra y se responde 401.

6.2 Validación en cada petición

El middleware `auth.js` verifica, además de la firma del token, el estado actual del usuario en la base:

1. La cuenta sigue activa.
2. El rol no cambió desde que se emitió el token.
3. La sesión no fue invalidada: `refresh_token_invalido_desde` se compara con el `iat` del token al segundo.
4. No tiene un cambio de contraseña pendiente. Si lo tiene, solo se permiten `/user/me` y `/user/me/change-password`; el resto responde 403 con `codigo: CAMBIO_CONTRASENIA_REQUERIDO`.

```
const invalidatedAfterIssue = user.refresh_token_invalido_desde &&
  decodedToken.iat < Math.floor(new Date(user.refresh_token_invalido_desde).getTime() / 1000)
if (invalidatedAfterIssue || user.id_rol !== decodedToken.id_rol) {
  return res.status(401).json({error: sessionInvalidatedMessage})
}
```

6.3 Cambio de contraseña obligatorio

Motivo	Cuándo ocurre	Pide la contraseña actual
`TEMPORAL`	Cuenta nueva o clave puesta por el administrador	No
`RECUPERACION`	Ingreso con código de verificación	No
`VENCIDA`	90 días o más desde el último cambio	Sí

Cambiar la contraseña reinicia el conteo de 90 días, limpia el motivo, invalida las demás sesiones y entrega un token nuevo al dispositivo que la cambió. Queda registrado en `Auditoria` como `CAMBIO_CLAVE`.

6.4 Límites de intentos

Operación	Límite	Observación
Inicio de sesión	10 fallos / 15 min por cuenta (IP + correo); 100 por IP	Los errores de un usuario no bloquean a los demás de la misma red; los ingresos correctos no cuentan
Envío de código	10 / 15 min por cuenta; 50 por IP	También aplica a la verificación de correo
Verificación de código	10 fallos / 15 min por cuenta; 50 por IP	Cada código se anula tras 5 intentos fallidos
Extracción de facturas	20 / hora por usuario	Protege la cuota del modelo de IA

7. Autorización

7.1 Por rol y por cargo

Cada ruta declara los roles habilitados con `roleMiddleware`; la restricción queda visible sin abrir el controlador:

```
router.post('/expense-review/:tripId/take',
  authMiddleware,
  roleMiddleware(['SUPERVISOR']),
  reviewController.takeExpenseReview)
```

Las rutas de tesorería usan `requireTreasurerPosition`, que valida el cargo en lugar del rol.

7.2 Asignación por jerarquía

Aprobador y revisor son roles únicos: todo viaje de su etapa les corresponde. El supervisor se resuelve con `hierarchyAssignmentService`: primero el jefe directo del empleado; si no lo hay, los supervisores de su sección; y si tampoco, cualquier supervisor. Un supervisor solo ve, toma u observa viajes de su alcance, los que tiene asignados o los que ya revisó.

7.3 Acceso a un viaje

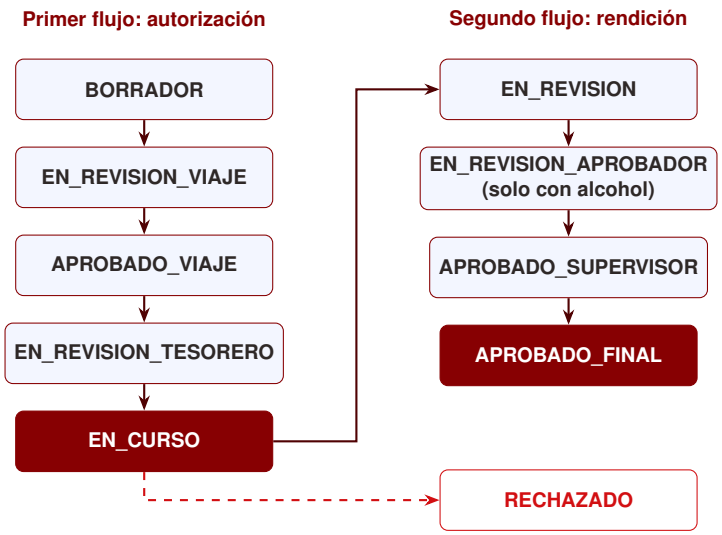
`tripAccessService` decide quién ve el detalle, los gastos y la planilla: el dueño, su reemplazo aprobado, el tesorero, el administrador, el aprobador, el revisor y el supervisor que corresponda. Un viaje inexistente responde 404 y uno sin permiso 403. Los recibos los emiten el dueño, su reemplazo o el tesorero.

7.4 Observaciones

`tripCommentService` solo permite observar a quien tiene la etapa actual del viaje. Las observaciones no se editan ni borran una vez decidida la etapa o en un ciclo anterior, y cada revisor solo gestiona las

suyas. Para rechazar se exige al menos una observación propia del ciclo vigente.

8. Flujos de aprobación



8.1 Reglas de transición

- Cualquier etapa de revisión puede rechazar el viaje. El rechazo incrementa `ciclo_revision` y `fue_iniciado` indica si ocurrió en el primer o en el segundo flujo.
- Un viaje rechazado antes de iniciarse se corrige editándolo y vuelve a `EN_REVISION_VIAJE`; uno rechazado en la rendición se corrige en sus gastos y se reenvía directamente a `EN_REVISION`.
- Las escrituras exigen el estado leído (`.eq('estado', X)`); si otra persona se adelantó se responde 409 y no se duplican correos ni registros.
- Si el responsable de una etapa única (aprobador, tesorero o revisor) es el propio viajero, la etapa se aprueba sola (`selfReviewSkipService`) y los documentos lo indican como aprobación automática. En el caso del tesorero, el fondo queda con el monto calculado al registrar el viaje.
- Cada aprobación o rechazo se registra en `Revision_Viaje`; las bandejas muestran en “Aprobados” lo que la persona aprobó y en “Rechazados” lo que rechazó y sigue rechazado, ordenado por la fecha de revisión más reciente.

9. Reglas de negocio

9.1 Monto del viaje

Al registrar o editar un viaje, el monto se calcula con las tarifas del cargo del empleado y se envía en `monto_asignado` y `monto_asignado_usd`. En un viaje nacional es *días × tarifa en Bs*. En uno interna-

cional, el día de salida y el de regreso se calculan con la tarifa en Bs y los días intermedios con la tarifa en USD. Tesorería revisa el monto y lo ajusta al asignar los fondos.

9.2 Control de gasto diario

El presupuesto se controla por día y por moneda. En un viaje nacional, lo gastado en bolivianos cada día no puede superar el `monto_diario` del cargo. En uno internacional, cada día se controla con las dos tarifas por separado: lo gastado en bolivianos contra `monto_diario` y lo gastado en dólares contra `monto_diario_usd`. Los hoteles quedan fuera del control diario y se comparan contra el total asignado. Cada día excedido, y el exceso en hoteles, exige una justificación en `Comentario.fecha_justificada`; las justificaciones con palabras inapropiadas se rechazan.

9.3 Moneda en viajes internacionales

En un viaje internacional se puede registrar gastos en bolivianos o en dólares cualquier día. Un gasto internacional se escribe en la moneda de su factura y su `monto_total` se guarda en dólares; los gastos internacionales no llevan impuestos.

9.4 Tipo de cambio

El gasto internacional trae `moneda` (USD, EUR, PEN, ARS, BRL, CLP, COP o MXN), el monto en esa moneda (`monto_moneda_origen`, o la suma de los subgastos, que van en la misma moneda), `tipo_cambio` (unidades por 1 USD, obligatorio si no es USD) y `tipo_cambio_bs` (bolivianos por 1 USD, siempre obligatorio). El backend calcula `monto_total` = monto ÷ `tipo_cambio`, redondeado a centavos, y en otra moneda deja la conversión en `Gasto_Tramo_Moneda`. La planilla en PDF va toda en Bs (`utils/expenseCurrency.js`): importe = `monto_total` × `tipo_cambio_bs`, con una columna MONTO ORIGINAL, una fila con el total internacional en USD para Oracle y saldos por fondo: un gasto internacional se descuenta del fondo en USD si el viaje lo tiene, o del fondo en Bs si solo se entregaron bolivianos. Un gasto antiguo sin `tipo_cambio_bs` se muestra en USD marcado “Sin T/C a Bs”.

9.5 Retenciones impositivas

Código	Tipo	Retenciones
`F`	Compra o servicio con factura	Ninguna; genera crédito fiscal de IVA (13 % del total)
`R`	Recibo: documento sin IVA	Ninguna; no lleva IVA ni crédito fiscal
`C`	Compra de bien sin factura	IUE 5 % e IT 3 % sobre la base incrementada (pagado ÷ 0.92)
`S`	Servicio o alquiler sin factura	RC-IVA 13 % e IT 3 % sobre la base incrementada (pagado ÷ 0.84)

En la factura (`F`) el IVA es el 13 % del total y se resta de él, igual que las retenciones: subtotal = total − IVA. Al empleado se le reconoce el total; en la planilla, en PDF y en Excel, el IVA va en su columna como crédito fiscal y el importe costo/gasto es el total menos el IVA (por ejemplo, 305.00 − 39.65 = 265.35). El recibo (`R`) no lleva IVA: subtotal y total son iguales, y la planilla en PDF y en Excel muestra IVA 0. En

los gastos sin factura la empresa asume los impuestos (base incrementada): el empleado registra lo que pagó; el RC-IVA o IUE y el IT se calculan cada uno con su porcentaje sobre lo pagado ÷ 0.84 en servicios (÷ 0.92 en compras), sin redondear esa división, y cada retención se redondea a centavos; la base es pagado + retenciones, como lo calcula contabilidad. Por ejemplo, con 50.00 pagados en un servicio el RC-IVA es 7.74, el IT 1.79 y la base 59.53. Si el gasto tiene subgastos, cada uno se calcula aparte y se suman, como en el recibo de papel: cuatro taxis de 50, 80, 60 y 50 dan 285.73, con RC-IVA 37.15 e IT 8.58. Calcular sobre el total de 240 de una vez daría 285.71, que no coincide con la suma de las líneas del recibo. Todos los importes se redondean a centavos con `roundCents`, que redondea bien los medios centavos (1.005 → 1.01), algo en lo que `toFixed` falla por la coma flotante. Lo pagado cuenta para el presupuesto y la base es el importe costo/gasto de la planilla. Los gastos internacionales quedan exentos. Los gastos con alcohol pierden toda retención y crédito fiscal y se imputan íntegros como costo.

9.6 Plazos de registro

Tras el fin del viaje hay cuatro días de tolerancia para cargar gastos. Vencido el margen, se debe pedir una ampliación al revisor, que otorga cuatro días desde su respuesta. El plazo es del viaje: la ampliación vale para el titular y para su reemplazo aprobado, cualquiera de los dos puede pedirla y la respuesta se avisa a ambos. El plazo no corre mientras el viaje está rechazado. La ampliación extiende el margen de carga, no el rango de fechas: la fecha del gasto debe pertenecer al periodo del viaje.

9.7 Rendición por terceros

El empleado puede pedir que otra persona activa de su sección rinda los gastos en su nombre. El revisor aprueba o rechaza; con la aprobación se avisa al titular y al reemplazo. El titular conserva siempre el acceso al viaje, y los documentos llevan su nombre. Si el viaje se envía a revisión con una solicitud pendiente de reemplazo o de plazo, esta se cierra automáticamente.

10. Catálogo de endpoints

Todas las rutas, salvo las de `/auth` y `/user/check-email`, requieren token de acceso. La columna “Acceso” indica la restricción adicional.

10.1 Autenticación y usuarios

Método	Ruta	Acceso	Descripción
POST	/auth	Público	Inicio de sesión
POST	/auth/refresh	Cookie	Renueva el token de acceso
POST	/auth/logout	Público	Cierra sesión y registra la salida
POST	/auth/send-code	Público	Envía el código de recuperación
POST	/auth/verify-code	Público	Valida el código e inicia sesión
GET	/user/me	Sesión	Datos del usuario actual
PUT	/user/me/update	Sesión	Actualiza el perfil

Método	Ruta	Acceso	Descripción
PUT	/user/me/photo	Sesión	Cambia la foto de perfil
PUT	/user/me/change-password	Sesión	Cambia la contraseña
GET	/user/employees	Roles	Lista de empleados para filtros
POST	/user/check-email	Público	Verifica si un correo está registrado
GET POST	/user	Administrador	Lista y crea usuarios
POST	/user/import/preview	Administrador	Revisa el Excel de usuarios sin guardar
POST	/user/import	Administrador	Crea los usuarios válidos del Excel
PUT DELETE	/user/:id	Administrador	Edita o elimina un usuario

La importación desde Excel (userImportService) admite hasta 500 filas y aplica las reglas del formulario. Marca los correos y CI que ya existen en el sistema o que se repiten en el archivo (todas las filas repetidas), los cargos y secciones que no existen, el rol inválido, los roles y cargos de una sola persona ocupados o repetidos, y el jefe directo que no está en el sistema ni en el archivo; si la fila del jefe tiene errores, la del subordinado tampoco se crea, y se detectan jefes en ciclo. Crea por niveles, primero los jefes, y cada usuario recibe en segundo plano su contraseña temporal por correo.

10.2 Viajes, gastos y facturas

Método	Ruta	Acceso	Descripción
GET	/trip/dashboard	Sesión	Viajes del panel del empleado
GET	/trip/history	Sesión	Historial paginado con filtro
GET	/trip/:id	Acceso al viaje	Detalle con gastos y resumen
GET	/trip/:id/rendicion-pdf	Acceso al viaje	Planilla de rendición en PDF
POST	/trip	Sesión	Crea un viaje
PUT	/trip/:id/edit	Dueño	Edita un borrador o un viaje rechazado
PUT	/trip/:id/submit-review	Dueño	Envía a revisión previa
PUT	/trip/:id/confirm-completion	Dueño o reemplazo	Finaliza y envía la rendición
GET	/trip/export/statements	Revisor, tesorero, administrador	Datos de las planillas de varios viajes para Excel
GET	/expense/:expenseId/detail	Acceso al viaje	Detalle de un gasto
POST	/expense/register	Dueño o reemplazo	Registra un gasto con su comprobante (imagen) y, solo en gastos internacionales, hasta 2 respaldos (respaldos)

Método	Ruta	Acceso	Descripción
PUT	/expense/:expenseId/update	Dueño o reemplazo	Edita un gasto; conserva los respaldos indicados en respaldos_conservados y agrega los nuevos, sin pasar de 2 y solo en gastos internacionales
DELETE	/expense/:expenseId	Dueño o reemplazo	Elimina un gasto
POST	/expense/:expenseId/receipt	Recibos	Envía el recibo individual
POST	/expense/trip/:tripId/receipt/:type	Recibos	Envía el recibo agrupado
POST	/invoice/extract	Sesión	Extrae datos de un comprobante
POST	/invoice/save	Dueño o reemplazo	Registra una factura
PUT	/invoice/:expenseId/update	Dueño o reemplazo	Edita una factura

10.3 Revisión y aprobación

Método	Ruta	Acceso	Descripción
GET	/review/trip-review/pending	Supervisor	Viajes en revisión previa sin asignar
GET	/review/trip-review/mine	Supervisor	Asignados e historial
POST	/review/trip-review/:tripId/take	Supervisor	Toma el viaje
POST	/review/trip-review/:tripId/return	Supervisor	Devuelve el viaje sin revisar
POST	/review/trip-review/:tripId/approve	Supervisor	Aprueba
POST	/review/trip-review/:tripId/reject	Supervisor	Rechaza con observaciones
GET	/review/expense-review/*	Supervisor	Mismas operaciones para la rendición
GET	/approver/pending-trips	Aprobador	Viajes por aprobar
POST	/approver/trip/:tripId/approve	Aprobador	Aprueba y genera el memorándum
POST	/approver/alcohol-review/:tripId/*	Aprobador	Revisión adicional por alcohol
GET	/treasurer/pending-trips	Tesorero	Viajes esperando fondos
PUT	/treasurer/:tripId/amounts	Tesorero	Ajusta los montos asignados
POST	/treasurer/:tripId/approve	Tesorero	Aprueba fondos y envía la confirmación
GET	/reviewer/pending	Revisor	Rendiciones para revisión final
POST	/reviewer/:tripId/approve	Revisor	Aprobación final y rendición en PDF
POST	/*/:tripId/comment	Etapa actual	Agrega, edita o borra observaciones

10.4 Solicitudes

Método	Ruta	Acceso	Descripción
POST	/deadline-authorization/trip/:tripId/request	Dueño o reemplazo	Pide ampliación de plazo
GET	/deadline-authorization/trip/:tripId/status	Dueño o reemplazo	Estado de la última solicitud
GET	/deadline-authorization/pending	Revisor	Solicitudes pendientes
POST	/deadline-authorization/:requestId/approve	Revisor	Aprueba (4 días desde la respuesta)
GET	/substitution/candidates	Sesión	Personas de la sección que pueden rendir
POST	/substitution/trip/:tripId/request	Dueño	Pide un reemplazo
GET	/substitution/mine	Sesión	Viajes que rinde como reemplazo
POST	/substitution/:requestId/approve	Revisor	Aprueba el reemplazo

10.5 Catálogos y administración

Método	Ruta	Acceso	Descripción
GET	/position, /section	Sesión	Cargos y secciones
POST PUT	/position, /section	Administrador	Crea y edita
PATCH	/position/:id/suspend	Administrador	Suspende o activa (también secciones)
GET	/expense-category	Sesión	Categorías de gasto, ordenadas por número de cuenta
GET	/role, /tax, /supplier	Administrador	Roles, impuestos y proveedores
GET	/audit	Administrador	Historial de accesos con filtros; las fechas son días de Bolivia
GET	/admin/dashboard	Administrador	Indicadores por fase; periodo opcional con fecha_inicio y fecha_fin
GET PUT	/admin/reminders	Administrador	Lee o guarda los días y horas del resumen
GET	/admin/reminders/preview	Administrador	A quién le llegaría el resumen, sin enviar
POST	/admin/reminders/send-now	Administrador	Envía el resumen en el momento

11. Servicios transversales

Servicio	Responsabilidad
`hierarchyAssignmentService`	Resuelve quién revisa cada etapa
`tripAccessService`	Permisos de lectura y de emisión de recibos
`deadlineService`	Valida fecha del gasto y plazo de carga con ampliaciones
`expenseSummaryService`	Gasto diario, exceso en hoteles y alertas
`alcoholDetectionService`	Detecta bebidas alcohólicas por gasto y por viaje: compara palabras completas, trata el combustible y sus abreviaturas de surtidor (G.ESPECIAL+) como inocuos y solo consulta a la IA los productos dudosos
`commentModerationService`	Rechaza términos prohibidos
`geminiService`	Modelo de IA con reintentos y modelo de respaldo
`reviewLogService`	Historial de revisión por persona y etapa
`selfReviewSkipService`	Aprobación automática de etapas propias
`auditLogService`	Registra ingresos, salidas y cambios de contraseña
`emailService`	Correo con plantilla institucional común
`pdfService`	HTML a PDF con JavaScript desactivado; el navegador se cierra siempre
`dailyDigestService`	Resumen de pendientes por rol; en modo prueba solo indica a quién le llegaría
`reminderScheduleService`	Guarda los días y horas del resumen y lo reprograma sin reiniciar
`tripExportService`	Reúne las planillas de varios viajes para exportarlas a Excel
`userImportService`	Revisa el Excel de usuarios y crea las filas válidas por niveles, primero los jefes

12. Documentos y notificaciones

12.1 Documentos PDF

Documento	Momento	Destinatario
Memorándum de aprobación	Aprobación del aprobador	Empleado
Confirmación de fondo	Aprobación de tesorería	Empleado (incluye origen y destino)
Recibos de compra y servicio	A pedido	Empleado; muestran la base incrementada como total, las retenciones y lo pagado como importe a cancelar; con subgastos, cada línea lleva su importe calculado por separado y el total es su suma; el número se reutiliza al reenviar

Documento	Momento	Destinatario
Rendición final	Aprobación final	Revisor y empleado, con saldo por moneda
Planilla de rendición	A pedido	Descarga del empleado

Todo texto escrito por usuarios se escapa (`htmlEscape`) antes de insertarse en correos y PDF. En el resultado final, cada moneda tiene su propia línea: en un viaje internacional puede corresponder devolver dólares y recibir un reembolso en bolivianos.

12.2 Correos inmediatos

- Al revisor: nueva solicitud de plazo o de reemplazo (indicando si la pidió el reemplazo).
- Al empleado: memorándum, fondo aprobado, resultado de la rendición, respuesta a sus solicitudes, bienvenida con contraseña temporal y código de recuperación.
- Al reemplazo: aviso de rendición asignada.

12.3 Resumen de pendientes

Un solo correo por persona con una línea por tipo de pendiente. Solo lo reciben quienes tienen algo pendiente; si falla el envío a una persona, se envía igual a las demás.

Programación configurable. Los días y las horas los define el administrador en la pantalla *Recordatorios*, que guarda la configuración en la tabla `Configuracion_Recordatorio` (una sola fila). `reminderScheduleService` crea un `node-cron` por cada hora, en la zona `America/La_Paz` sin importar la del servidor, y al guardar detiene los anteriores y crea los nuevos, sin reiniciar el servidor.

- Se puede activar o desactivar el envío.
- Días de la semana a elección (`0` domingo a `6` sábado) y hasta 4 horas por día en formato `HH:MM`.
- Configuración de fábrica: de lunes a viernes a las 08:00, 12:00 y 16:00.
- Si la tabla no tiene la fila, el servidor usa la configuración de fábrica.

Rol	Contenido
Supervisor	Viajes en revisión previa y rendiciones, solo de su alcance
Aprobador	Viajes por aprobar y rendiciones con alcohol
Revisor	Rendiciones finales y solicitudes de plazo y reemplazo vigentes
Tesorero	Viajes esperando asignación de fondos

```
npm run resumen -- --prueba   # muestra destinatarios y contenido sin enviar
npm run resumen               # envía los correos en el momento
# o desde la pantalla Recordatorios: Vista Previa y Enviar Ahora
```

13. Extracción de datos de comprobantes

La imagen del comprobante se envía al modelo de visión de Gemini (`invoiceExtractionService`) con una instrucción que fija el contexto tributario boliviano: el "Importe" es el total con IVA incluido, el IVA es del 13 % y debe devolverse como monto en dinero. El modelo responde con un JSON que contiene proveedor, NIT, CUF (código de autorización), número de factura, fecha de emisión, monto sin impuestos, IVA, total, tipo de documento (`F` o `R`) y detalle de productos.

- `geminiService` reintenta ante errores temporales y, si el modelo principal falla o agotó su cuota, usa un modelo de respaldo.
- La respuesta se normaliza: los datos que faltan quedan como "No Especificado", la fecha se valida (`fecha_emision_valida`) y la cantidad de cada producto se redondea a 3 decimales (por ejemplo 53.345 litros de combustible) y su precio unitario a 4 (por ejemplo 3.7456). En un recibo el monto es el total pagado y el IVA es 0. En una factura, el IVA es el 13 % del total y se resta de él (subtotal = total − IVA); si la IA devuelve otros valores, se recalculan así sin cambiar el total. Si un recibo o boleto no tiene número, se usa su código, que puede tener símbolos como `@` o `\protect \T1\textdollar .` El CUF se guarda solo con letras y números en mayúsculas (`utils/cuf.js`) y solo en facturas; el recibo no lo lleva.
- Al registrar, el empleado revisa y puede corregir todos los datos antes de guardar, incluida la fecha de emisión por si la IA la leyó mal; al modificar una factura ya guardada, la fecha queda bloqueada. El gasto se marca como `modificado` cuando hubo correcciones.
- Al guardar, cada producto debe tener nombre, un precio válido con hasta 4 decimales y una cantidad mayor a cero con hasta 3 decimales, igual que las columnas de `Detalle_Factura`: `precio` es `numeric(12, 4)` y `cantidad` es `numeric(12, 3)`.

Las imágenes (el comprobante y, en gastos internacionales, hasta 2 respaldos; en un gasto en bolivianos se rechazan con 400) se suben al almacenamiento (`storageService`) antes de escribir el gasto o la factura. Los archivos que dejan de usarse se borran: al eliminar un gasto, al reemplazar o quitar un comprobante, respaldo o foto de perfil, y los recién subidos si falla el guardado; `npm run limpiar-archivos` borra los que hayan quedado sueltos (con `-- --prueba` solo los lista). El bucket es público y el backend usa `service_role`, así que no hacen falta políticas sobre `storage.objects`; más archivos de los permitidos se rechazan con un mensaje claro; si la subida falla no queda nada a medio guardar. Una factura repetida (número, fecha y proveedor) se rechaza antes de modificar datos.

14. Seguridad

Aspecto	Medida
Contraseñas	bcrypt; mínimo 8 caracteres; vencen a los 90 días
Sesión	Token corto en memoria, renovación en cookie `httpOnly`; invalidación inmediata por cambio de rol, suspensión o cambio de contraseña
Ingreso	Mismo mensaje si el correo no existe o la clave es incorrecta
Orígenes	CORS restringido a los dominios del cliente

Aspecto	Medida
Proxy	`trust proxy` solo en Render o con `TRUST_PROXY=true`, para no falsear la IP
Datos de usuarios	Escapados en correos y PDF; nunca se devuelve el hash de la contraseña
Concurrencia	Escrituras condicionadas al estado leído; 409 si otra persona se adelantó
Base de datos	RLS activo; la clave de servicio solo existe en el servidor
Importación de usuarios	Solo el administrador; hasta 500 filas; el servidor vuelve a validar todo aunque el navegador ya mostró la vista previa
Peticiones	Cuerpo JSON de hasta 1 MB; las imágenes van aparte, con su propio límite

15. Despliegue

15.1 Piloto en Render

El backend se publica como servicio web de Node con `npm start`. Render se detecta solo (`RENDER=true`), por lo que no hace falta `TRUST_PROXY`. En el plan gratuito el servicio se suspende tras unos minutos sin uso, y el resumen de pendientes puede no enviarse a su hora; para el uso definitivo se recomienda un VPS.

15.2 Servidor VPS

1. Instalar Node.js 22.12 o superior, clonar el repositorio y ejecutar `npm ci`.
2. Instalar las librerías del sistema que necesita Chromium para generar los PDF.
3. Crear el `.env` con `NODE_ENV=production` y `TRUST_PROXY=true`.
4. Mantener el proceso con `pm2` en una sola instancia, para que el resumen no se envíe duplicado.
5. Publicar detrás de Nginx con HTTPS (Let's Encrypt), `client_max_body_size` de 15 MB y tiempo de espera de 120 s.
6. Cerrar el puerto del backend con el firewall; solo Nginx lo usa.
7. Apuntar `VITE_API_URL` del cliente al nuevo dominio.

```
pm2 start src/index.js --name scv-backend -i 1
pm2 save && pm2 startup
sudo certbot --nginx -d api.tuempresa.com
```

16. Dependencias y seguridad

`npm audit` no reporta vulnerabilidades. Puppeteer se actualizó de la versión 10 a la 25, que ya no arrastra dependencias vulnerables; los ocho documentos PDF se generaron antes y después con los mismos datos y salen iguales (el tamaño A4 ahora es exacto). No hace falta `npm audit fix --force`.

17. Pruebas automáticas

Una prueba automática es código que usa el sistema como lo haría una persona y verifica que el resultado sea el correcto. Se ejecutan con un comando y, si un cambio rompe algo, fallan antes de que llegue a producción.

Tipo	Qué prueba	Comando
Unitarias	Funciones sueltas: retenciones, IVA de factura y recibo en la planilla, cantidades con decimales, control diario por moneda, exportación, recordatorios, jerarquía, moderación, retenciones por subgasto y validación de la importación de usuarios (`tests/services/`)	`npm test`
De la API	Endpoints completos contra una base de prueba: ingreso, permisos por rol, monto del viaje, gastos en ambas monedas, gasto internacional en soles pasado a USD y tipo de cambio a Bs obligatorio, número de recibo del sistema, recibo sin IVA con cantidad decimal, CUF de la factura, respaldos solo en gastos internacionales, presupuesto con lo pagado, número de rendición, códigos de Oracle, exportación e importación de usuarios desde Excel (`tests/api/`)	`npm run test:api`

Las pruebas de la API necesitan una base PostgreSQL de prueba creada con los scripts de `database/` (`01` a `04` y `testing_02_organizacion_prueba.sql`), servida por PostgREST y por el proxy `tests/support/supabaseProxy.js`, que imita la ruta `/rest/v1` de Supabase y acepta las subidas a Storage sin guardarlas y anota los archivos que se piden borrar; nunca se ejecutan contra la base real. En el estado actual pasan 74 pruebas unitarias y 36 de la API.

17.1 Integración continua

El archivo `.github/workflows/pruebas.yml` ejecuta las pruebas en GitHub Actions en cada envío y en cada pull request: un trabajo corre las unitarias y otro levanta una base PostgreSQL temporal, PostgREST y el proxy, y corre las de la API. El resultado aparece como aprobado o fallido junto a cada commit, en la pestaña *Actions* del repositorio; si una prueba falla, GitHub la marca junto a la línea de código que falló. En las pruebas, los correos no se envían de verdad: se simulan y se revisa a quién se habrían mandado.

18. Convenciones

- El código se escribe en inglés; los textos visibles y el modelo de datos, en español.
- Cada función del backend lleva un comentario de una línea con su propósito; dentro de las funciones solo se dejan avisos importantes, también de una línea.
- Punto y coma al final de cada sentencia y literales de objeto sin espacios internos (`{dato: valor}`).
- Los scripts SQL llevan comentarios breves: un encabezado por sección y una línea donde haga falta.
- Se prefieren bloques `if`/`else` explícitos a los operadores ternarios en la lógica.
- Los commits siguen Conventional Commits: `feat`, `fix`, `style`, `refactor`, `docs`, `test` y `chore`.